
OBJECT-ORIENTED PROGRAMMING

Python

Object-oriented programming

■ First goal: Model the objects of the world

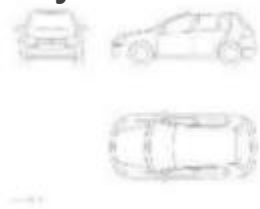
- Noun-oriented
- Focus on the domain of the program

■ Phases

- Object-oriented analysis: Understand the domain
 - Define an object-based model of it
- Object-oriented design: Define an implementation
 - Design the solution
- Object-oriented programming: Build it

Object-oriented programming

- Object in Python is a representation of a person, a place, a bank account, a car, or any item that the program should handle
- OOP Recipe:
 - (1) define **classes** (these are descriptions)
 - (2) **make object instances** out of classes



Class Car



Car instances

Object-oriented programming

Creating a simple class in Python

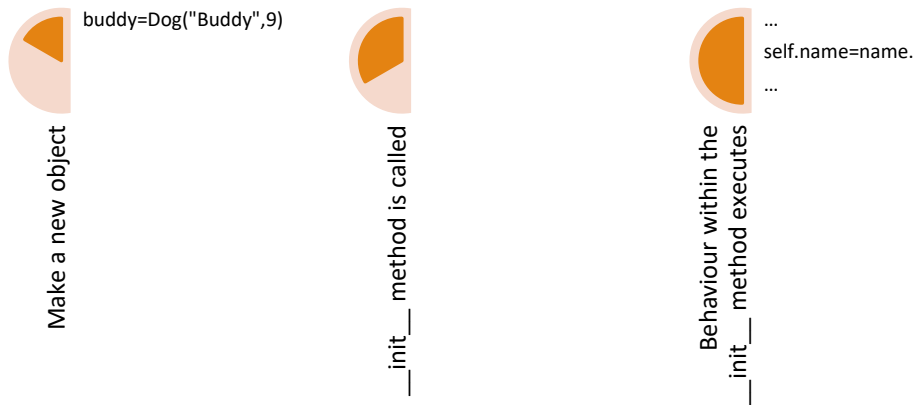
- In a class, we describe (1) how the object will look like, and (2) what behavior it will have.
- Classes are the blueprints for a new data type in your program
- A class should have at minimum:
 - **A name** (e.g. Class Person)
 - **A constructor method** (that describes how to create a new object, `__init__`)

```
class Dog:
    species = "Canis familiaris"
    def __init__(self, name, age):
        self.name = name
        self.age = age
```

Object-oriented programming

The `__init__` method

- `__init__` is a special method in python classes
- The `__init__` method is the constructor method for a class
- `__init__` is called when ever an object of the class in constructed



Object-oriented programming

Creating an object from the class

- From one class, you make objects (instances)

```
class Dog:
    species = "Canis familiaris"
    def __init__(self, name, age):
        self.name = name
        self.age = age
```

```
buddy = Dog("Buddy", 9)
print(buddy.name)
```

Buddy

Object-oriented programming

Behavior of a class

- Classes/object can have methods just like function except that they have an extra self variable at the beginning
- An object method takes as the first parameter the object (self) and can accept any number of other parameters

```
class Dog:
    species = "Canis familiaris"
    def __init__(self, name, age):
        self.name = name
        self.age = age
    # Instance method
    def description(self):
        return f"{self.name} is {self.age} years old"

    # Another instance method
    def speak(self, sound):
        return f"{self.name} says {sound}"
```

Object-oriented programming

Object vs Class Variables

- Most variables are object specific (for example, the variable number of passengers in a taxi is different for every taxi that we create)
- Some variables are common to all objects (for example, if we want to count the number of taxis that we have in our company)
- To access a class variable within a method, we use `@classmethod` decorator, and pass the class to the method

Object-oriented programming

Access Class Variables

- To access a class variable within a method, we use @classmethod decorator, and pass the class to the method

```
class Taxi:
    numberOfTaxi = 0
    def __init__(self, driver_name, on_duty, cities):
        self.dname = driver_name
        self.oduty = on_duty
        self.cities = cities
        self.num_passengers = 0
        Taxi.numberOfTaxi = Taxi.numberOfTaxi + 1

    @classmethod
    def how_many(cls):
        """return the current Taxi inventory"""
        return cls.numberOfTaxi
```

```
first_tx = Taxi('Tien', True, ['HN', 'BN'])
second_tx = Taxi('Bao', False, ['HCM', 'CM'])
```

```
print(f'{first_tx.dname} is driving {first_tx.num_passengers} passengers. Currently, we have: {Taxi.how_many()} taxis' )
```

```
Tien is driving 0 passengers. Currently, we have: 2 taxis
```

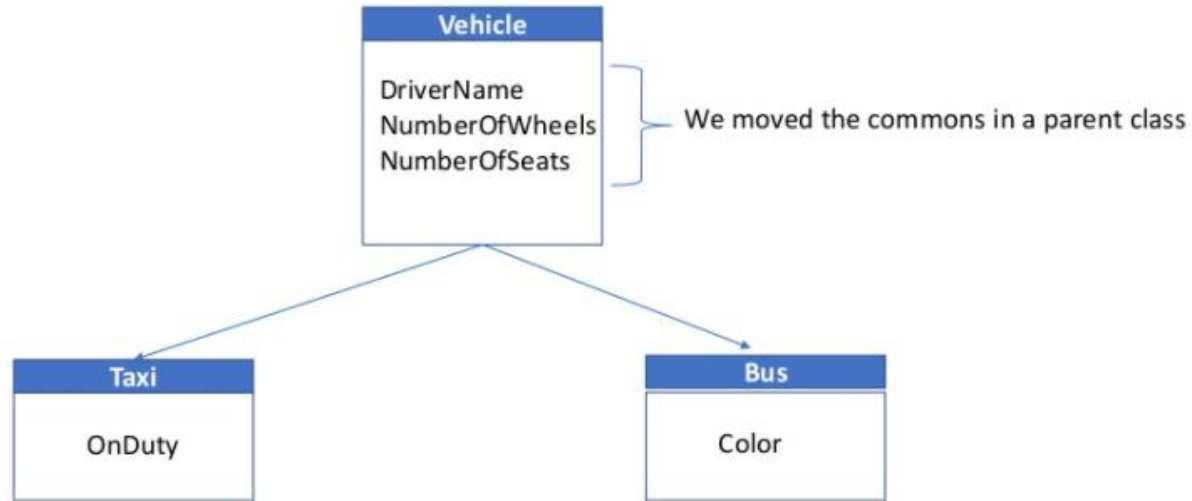
Object-oriented programming

Inheritance

- Inheritance simplifies our code through reuse of the code that has been already written.
- Inheritance is a relation between a parent class (e.g. Vehicle) and children classes (e.g. Taxi, Bus, Truck, etc)
- A class inherits attributes and behavior methods from its parent classes

Object-oriented programming

Inheritance example



The child classes only keep the attributes and methods relevant to them

Object-oriented programming

Inheritance in Python

- Create a parent class (e.g. Vehicle) with the common attributes and common methods.
- Create child classes (e.g. Bus and Taxi) with the extended attributes and extended methods,
 - Pass the class definition to the child (e.g **Class Bus(Vehicle):**)
 - Use the parent attributes and method through **super()**.

Object-oriented programming

Inheritance in Python

```
class Vehicle:
    def __init__(self, driver_name, number_of_wheels, number_of_seats):
        self.dname = driver_name
        self.nwheels = number_of_wheels
        self.nseats = number_of_seats
```

```
class Taxi(Vehicle):
    def __init__(self, driver_name, number_of_wheels, number_of_seats, on_duty):
        super().__init__(driver_name, number_of_wheels, number_of_seats)
        self.oduty = on_duty
```

```
class Bus(Vehicle):
    def __init__(self, driver_name, number_of_wheels, number_of_seats, color):
        super().__init__(driver_name, number_of_wheels, number_of_seats)
        self.color = color
```

Object-oriented programming

Method Overriding

- Method overriding is an object-oriented programming feature that allows a subclass to provide a different implementation of a method that is already defined by its superclass or by one of its superclasses.
- `__init__` in the child class (e.g. Taxi) overrides the `__init__` method from the parent class

Object-oriented programming

Example of Overriding `__str__`

- Let's add a `__str__` method that nicely prints our Vehicle details on the screen

```
class Vehicle:
    def __init__(self, driver_name, number_of_wheels, number_of_seats):
        self.dname = driver_name
        self.nwheels = number_of_wheels
        self.nseats = number_of_seats
    def __str__(self):
        return f'This vehicle is driven by: {self.dname} and it has {self.nwheels} wheels.'
```

```
vehicle_1st = Vehicle('Tien', 4, 4)
print(vehicle_1st)
```

This vehicle is driven by: Tien and it has 4 wheels.

Object-oriented programming

Example of Overriding __str__

```
class Vehicle:
    def __init__(self, driver_name, number_of_wheels, number_of_seats):
        self.dname = driver_name
        self.nwheels = number_of_wheels
        self.nseats = number_of_seats
    def __str__(self):
        return f'This vehicle is driven by: {self.dname} and it has {self.nwheels} wheels.'
```

```
class Taxi(Vehicle):
    def __init__(self, driver_name, number_of_wheels, number_of_seats, on_duty):
        super().__init__(driver_name, number_of_wheels, number_of_seats)
        self.oduty = on_duty
    def __str__(self):
        return f'Taxi: This vehicle is driven by: {self.dname} and it has {self.nwheels} wheels.'
```

```
taxi_1st = Taxi('Tien tx', 4, 2, True)
print(taxi_1st)
```

Taxi: This vehicle is driven by: Tien tx and it has 4 wheels.

Object-oriented programming

Object-relational mapping (ORM)

- ORMs help you a lot in writing complex CRUD operations, which are a pain to write via manual SQL
 - Create a new object in a database
 - Update an object in a database
 - Delete an object from database
- We will use peewee in this course to try-out object-relational mapping

Object-oriented programming

ORM Approach with peewee

1. Import peewee
2. Set the database name
3. Create the class for the Vehicle by inheriting from peewee.Model
4. Create the table for the class
5. Create a few instances of the Vehicle
6. Save them in the database

Object-oriented programming

ORM Approach with peewee

1. Import peewee

2. Set the database name

```
# We first import the ORM
import peewee as pw

# We set the name of the Sqlite database
db = pw.SqliteDatabase('vehicle.db')
```

Object-oriented programming

ORM Approach with peewee

3. Create the class for the Vehicle by inheriting from peewee.Model

```
class Vehicle(pw.Model):
    """
    ORM model of the Vehicle
    """
    dname = pw.TextField()
    nwheels = pw.IntegerField()
    nseats = pw.IntegerField()

    class Meta:
        '''
        we connect the database to the models via the nested class
        '''
        database = db
```

Object-oriented programming

ORM Approach with peewee

Field types table

Field Type	Sqlite	Postgresql	MySQL
<code>AutoField</code>	integer	serial	integer
<code>BigAutoField</code>	integer	bigserial	bigint
<code>IntegerField</code>	integer	integer	integer
<code>BigIntegerField</code>	integer	bigint	bigint
<code>SmallIntegerField</code>	integer	smallint	smallint
<code>IdentityField</code>	not supported	int identity	not supported
<code>FloatField</code>	real	real	real
<code>DoubleField</code>	real	double precision	double precision
<code>DecimalField</code>	decimal	numeric	numeric
<code>CharField</code>	varchar	varchar	varchar
<code>FixedCharField</code>	char	char	char
<code>TextField</code>	text	text	text

Object-oriented programming

ORM Approach with peewee

4. Create the table for the class

```
try:
    Vehicle.create_table()
except pw.OperationalError:
    print('Vehicle table already exists')
```

Object-oriented programming

ORM Approach with peewee

5. Create a few instances of the Vehicle

6. Save them in the database

```
vh_1 = Vehicle(dname = 'Tien A', nwheels=4, nseats=5)
vh_2 = Vehicle(dname = 'Tien B', nwheels=4, nseats=6)
vh_3 = Vehicle(dname = 'Tien C', nwheels=8, nseats=7)

vh_1.save()
vh_2.save()
vh_3.save()
```

Object-oriented programming

Other peewee things

- You can query for items in a database
- You can update item using `.save()`
- You can set primary and foreign keys
-

Object-oriented programming

Other peewee things

- You can query for items in a database

```
Vehicle.get(Vehicle.dname=='Tien A').dname
```

```
'Tien A'
```

```
vehicles = Vehicle.select().where(Vehicle.dname.contains('B'))  
vehicles.get().dname
```

```
'Tien B'
```

```
vehicles = Vehicle.select().where(Vehicle.dname.in_(['Tien A', 'Tien B']))  
vehicles.get().dname
```

```
'Tien A'
```

Thank you
